1. Đặt ra vấn đề:

Bảo muốn tạo tài khoản Facebook mới, nhưng vì thiếu kinh nghiệm trong việc đặt mật khẩu, Bảo đã đặt một mật khẩu cực kì dễ đoán.

Bảo: Chắc đặt tên là top1bilac123, mật khẩu chắc đặt 123456789, chắc là sẽ không ai hack tài khoản mình đâu

Một lúc sau…

Thịnh: Chà, tài khoản ai đây, top1bilac123 à. Chắc phải hack để lụm 67k rau má thôi. Để đoán thử, hmmm, từ 1 đến 9 thử đi. Ngon, được luôn.

Bảo nhận được cuộc điện thoại.

Thịnh: Alo bảo à bảo.

Bảo: không phải anh ơi. Em không phải bảo.

Thịnh: Bảo đừng có chối, bảo không nói chuyện nghiêm túc với anh là anh tung hết nội dung tin nhắn Facebook của bảo lên trên mạng đó.

Bảo: Đừng mà anh ơi, em còn mẹ già con thơ ở nhà nữa.

Thịnh: Thế chuyển anh 67k rau má thì anh trả lại tài khoản, không anh đăng hết lên trên mạng đó.

Bảo: Anh đợi em tí anh ơi, em chuyển liền.

Theo các bạn đã thấy, bạn Bảo do thiếu kinh nghiệm, thông tin trong việc đặt mật khẩu mạnh mà đã gây ra những hậu quả, ảnh hưởng đến chính bản thân Bảo. Như vậy, mật khẩu mạnh là gì? Làm sao để có mật khẩu mạnh,...

Mật khẩu mạnh là gì?

Thông thường, mật khẩu mạnh là một mật khẩu khó đoán, khó bị các hacker bẻ khóa và dễ nhớ đối với người dùng. Các mật khẩu mạnh thường dài tối thiểu 12 ký tự và có chứa chữ hoa, chữ thường, số, cùng các ký tự đặc biệt.

Làm sao để có mật khẩu mạnh:

Để có thể biết được mật khẩu của mình có mạnh hay không, và tạo mật khẩu mạnh hơn dựa trên mật khẩu mà người dùng nhập thì mời các bạn đến với dự án của chúng mình.

---

2. Giới thiệu chung:

3 chức năng chính: đánh giá mật khẩu, tính thời gian bẻ khóa, gợi ý mật khẩu

---

Đánh giá mật khẩu:

thư viện zxcvbn là một thư viện mã nguồn mở, nó tính độ khó đoán của mật khẩu bằng cách nhận diện các mẫu như từ trong từ điển, tên người, tên phổ biến; chuỗi phím liền kề; ngày tháng năm sinh; từ lặp lại hoặc chuỗi số => sau phân tích, thư viện sẽ trả về điểm 0 - 4, từ rất yếu đến rất mạnh, kèm theo thời gian bẻ khóa.

Sử dụng thư viện zxcvbn được tạo bởi **Dropbox**: tiện lợi do đánh giá mật khẩu dựa trên từ điển, tên riêng, chuỗi ngày tháng, các phím liền kề mà không đánh giá dựa trên các quy tắc cứng nhắc như bắt buộc có số, kí tự đặc biệt, chữ hoa, chữ thường

Tại sao không chọn những thư viện khác:

Dùng quy tắc lỗi thời: bắt buộc phải có kí tự, số, chữ hoa, …

Có thể do một nhóm nhỏ làm => không đáng tin, không an toàn = zxcvbn

---

Tính thời gian bẻ khóa:

thư viện zxcvbn ước lương thời gian bẻ khóa mật khẩu dựa trên việc **phân tích cấu trúc mật khẩu theo hành vi người dùng**

1. Ý tưởng chính

dựa trên nhận định : người dùng thường tạo mật khẩu theo các mẫu dễ đoán , không phải ngẫu nhiên hoàn toàn

từ đó:

- phân tách mật khẩu thành các cụm quen thuộc
- ước lượng số lần đoán cần thiết cho từng cụm
- kết hợp lại để tính thời giản bẻ khóa

2. Các bước của thuật toán

Bước 1 : phân tích và nhận diện các cụm

quét qua mật khẩu để tìm các cụm như:

- **Từ điển** : password , hello , deptrai
- **Dãy số** : 1234 , 9876
- **Dãy bàn phím** : zxcvbn , qwert
- **Lặp lại** : aaa , abcabc
- **Ngày tháng** : 20062010

Ví dụ :

tuideptrai123!

→ “tuideptrai” + “123” + “!”

Bước 2 : gán số lần đoán

mỗi cụm được gán 1 giá trị

- **Từ phổ biến** → số lần đoán nhỏ
- **Từ hiếm** → số lần đoán lớn
- **Chuỗi ngẫu nhiên** → rất lớn

Ví dụ :

- “password” → ~10
- “hello” → ~ 1000
- chuỗi ngẫu nhiên → ~ 1000000 +

Bước 3 : tối ưu phân tách

Một mật khẩu mạnh có thể được chia theo nhiều cách khác nhau

thư viện zxcvbn sẽ dùng **quy hoạch động** để:

- Duyệt tất cả các cách phân tách
- chọn cách có **tổng số lần đoán nhỏ nhất**

→ Đây là trường hợp tấn công hiệu quả nhất được nhắm tới

Bước 4 : tính tổng số lần đoán

Tổng số lần đoán được tính bằng cách kết hợp các cụm

tổng số lần đoán = số lần đoán của từng cụm cộng lại

Bước 5 : quy đổi sang thời gian

Từ số lần đoán , zxcvbn ước lượng thời gian theo công thức :

thời gian bẻ khóa = số lần đoán / tốc độ thử mật khẩu

Các cách thường dùng :

- online (bị giới hạn)
- offline với hash chậm
- offline với hash nhanh

Làm sao để đoán được:

Dựa vào từ điển tần suất được nhúng sẵn: thư viện zxcvbn có một từ điển các mật khẩu bị lộ và được sắp xếp theo tần suất xuất hiện trong các vụ lộ mật khẩu thực tế. Chẳng hạn như mật khẩu “password” dùng nhiều thứ nhì, nên đứng thứ nhì trong từ điển, nên được gán số lần đoán là 2; mật khẩu “batman” đứng ở vị trí 45 thì số lần đoán là 45. Và thư viện cũng giả định là các hacker cũng sẽ dùng từ điển y hệt để bẻ khóa.

Dựa vào toán tổ hợp:

nếu bạn nhập “H3llo thì sẽ tính như sau, với ở đây giả dụ hello là ở vị trí thứ 112 trong từ điển:
Bước 1: Tra từ gốc hello -> mất 101 lần đoán

Bước 2: Tính số cách viết hoa, do thói quen thường ghi hoa ở chữ cái đầu nên thư viện zxcvbn nhận ra -> mất 2 lần đoán

Bước 3: tính **số cách thay thế** chữ e bằng số 3 -> mất 2 lần đoán

->Công thức: Tổng số lần đoán = thứ hạng từ gốc(112) x hệ số viết hoa x hệ số cách viết chữ hoa = 101 x 2 x 2 = 404

Gợi ý mật khẩu:

Sử dụng AI để xử lí mật khẩu của người dùng, sau đó đưa mật khẩu mạnh hơn cho người dùng.
Công cụ hỗ trợ: API key

Bước 1: Input (Người dùng nhập liệu)

- **Hành động:** Bạn gõ mật khẩu yếu của mình vào phần mềm (ví dụ: conmeo).
- **Phần mềm chuẩn bị:** Phần mềm trên máy bạn sẽ không gửi chữ conmeo trơ trọi. Nó sẽ chữ này vào một câu lệnh rõ ràng, ví dụ: Hãy tạo một mật khẩu mạnh từ chữ “con mèo”, thêm số và ký tự đặc biệt.

Bước 2: Đóng gói và đính kèm API Key (Client-side)

- Lúc này, phần mềm sẽ tạo ra một gói tin điện tử, thường ở định dạng JSON.
- Trong gói tin này có hai thành phần quan trọng nhất:
    1. **Dữ liệu (Payload):** Câu lệnh chứa chữ `conmeo` ở Bước 1.
    2. **Chứng minh thư (API Key):** Mã API Key của bạn sẽ được gắn vào phần tiêu đề của gói tin.

Bước 3: Di chuyển qua Internet (Network Request)

- Phần mềm ném gói tin này lên đường truyền Internet, gửi thẳng đến địa chỉ máy chủ của nhà cung cấp AI, như máy chủ của Google. Tất cả dữ liệu này đều được mã hóa trên đường truyền để không ai đọc trộm được.

Bước 4: Máy chủ tiếp nhận và Xác thực (Server-side Authentication)

- **Kiểm tra API Key:** Khi gói tin đến nơi, máy chủ sẽ kiểm tra phần tiêu đề trước tiên.
    - *Nếu* API Key sai hoặc hết hạn*:* Nó từ chối ngay lập tức và báo lỗi Không có quyền truy cập.
    - *Nếu* API Key đúng*:* Người gác cổng mở cửa cho gói tin đi tiếp, đồng thời ghi lại vào sổ sách: “Tài khoản sở hữu API Key này vừa dùng 1 lượt yêu cầu”.

Bước 5: AI Xử lý (AI Processing)

- Sau khi qua cửa, câu lệnh được đưa vào lõi của Mô hình AI (như Gemini).
- AI đọc hiểu yêu cầu và dùng thuật toán của mình để biến đổi `conmeo` thành một thứ phức tạp hơn, ví dụ: `C0nM3o@!2024`.

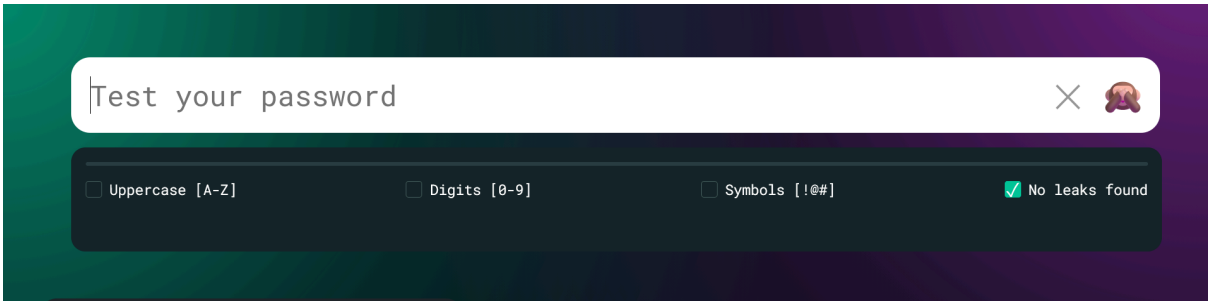
Bước 6: Output (Trả kết quả về)

- Máy chủ Google đóng gói chữ `C0nM3o@!2024` này vào một gói tin phản hồi và gửi ngược lại qua Internet về máy tính của bạn.
- Phần mềm của bạn nhận gói tin, bóc tách lấy đúng chuỗi mật khẩu và hiển thị lên màn hình cho bạn xem.

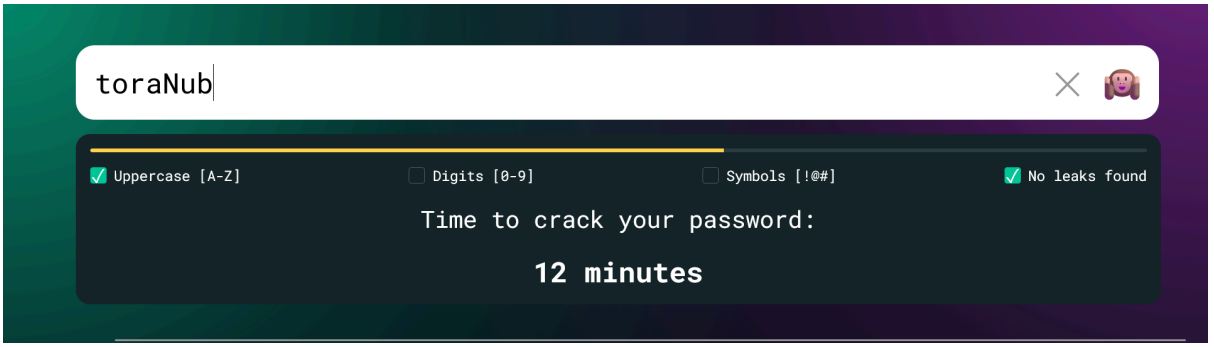
---

2. Giới thiệu sơ qua trang web:
Sử dụng html, css, js để tạo nên trang web.
Hình ảnh tổng quan cho web: ….

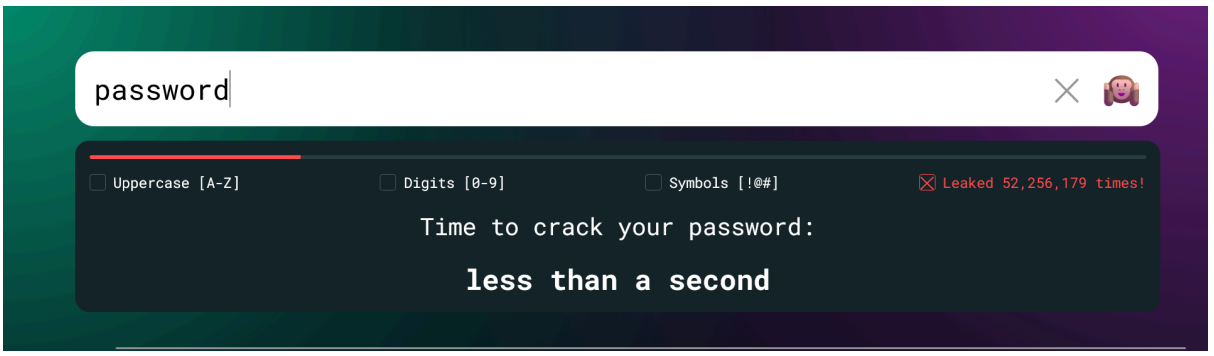
Phần kiểm tra mật khẩu:



Ô để nhập mật khẩu cần kiểm tra độ mạnh, có các ô thể hiện có chữ in hoa, kí tự đặc biệt và phần hiển thị số lần lộ mật khẩu



Ví dụ, khi nhập mật khẩu “toraNub” vào thì ô uppercase sẽ được đánh dấu và thời gian tính toán là 12 phút.



Hình ảnh ví dụ thể hiện số lần bị lộ

Phần tạo mật khẩu:

Password Generator

Base Word (Optional):

e.g. dragon

Type a word to enhance, or leave blank for a random phrase!

Target Length: 16

- [x] Uppercase
- [x] Numbers
- [x] Symbols
- [ ] Avoid Tricky (I, l, O, 0)

Generate

GIới thiệu sơ qua giao diện, nút,...

Ưu điểm:

- Hỗ trợ người dùng đánh giá độ mạnh của mật khẩu
- Hỗ trợ người dùng tạo mật khẩu mạnh hơn
- Giúp người sử dụng mạng xã hội, các trang web nâng cao nhận thức bảo mật (có thể ghi tắc thành nâng cao nhận thức bảo mật)
- Giúp bảo mật thông tin của người dùng

Nhược điểm:

- Dataset để kiểm tra độ mạnh của password không rộng lớn -> Tính chính xác không đảm bảo
- Mật khẩu gợi ý từ web chưa thực sự dễ nhớ, thân thiện với người dùng